Build and Deploy a Small Web App

A hands-on Reading List tutorial

This guide walks through a small app that saves books, marks them as read, and lets you remove them. It is deliberately much smaller than the apps in the reference repositories, but it connects the same core layers:

- **Frontend:** React 19, TypeScript, and Vite
- **API:** ASP.NET Core 8 Web API
- **Data:** PostgreSQL 16 with Entity Framework Core 8
- **Hosting:** Netlify for the frontend; Render for the API and database

You do not need to add accounts, authentication, or AI for this project. The aim is to get one useful feature working end to end, then deploy it.

The examples follow the documented architecture in `Jeemzu/jeemzu.me` and `Jeemzu/jeemzu.api`. Those are good places to look if you want to see a larger version of the same stack.

1. Get the tools ready

Install:

- **Node.js 20.19+** (or a current Node.js LTS release) and npm
- **.NET 8 SDK**
- **Docker Desktop** or Docker Engine, to run PostgreSQL locally
- **Git** and a GitHub account

The shell commands below use a Bash-style terminal (macOS/Linux Terminal, Git Bash, or WSL). If you use PowerShell, enter multi-line commands as one line or adapt the line continuations.

2. Create the frontend and API

Keep the projects in separate folders, like the frontend and API repositories:

```
mkdir reading-list
cd reading-list
dotnet new webapi --use-controllers --framework net8.0 -n ReadingList.Api
npm create vite@latest reading-list-web -- --template react-ts
```

Install and start the frontend:

```
cd reading-list-web
npm install
npm run dev
```

Open the address Vite prints, usually `http://localhost:5173`. You should see the starter page. Stop the server with `Ctrl+C`.

3. Start PostgreSQL on your computer

This runs PostgreSQL 16 in Docker and keeps its data in a named volume:

```
docker run --name reading-list-db -e POSTGRES_USER=readinglist -e
POSTGRES_PASSWORD=local_dev_password -e POSTGRES_DB=readinglist -p 5432:5432 -v reading-list-db-
data:/var/lib/postgresql/data -d postgres:16
```

The database is available locally on port `5432`. This password is only for your own development database; make a different password for production.

To stop the database later, run `docker stop reading-list-db`. Start it again with `docker start reading-list-db`. If Docker says the container name is already in use, the existing container is probably still there.

4. Set up EF Core and a private connection string

From the `reading-list` folder:

```
cd ReadingList.Api
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 8.*
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.*
```

Install the EF command-line tool if you do not already have it:

```
dotnet tool install --global dotnet-ef --version 8.*
```

Use .NET user secrets for the local database connection. They live outside the project, so they will not be committed with your code:

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "<paste your local Npgsql connection
string>"
```

Replace the placeholder with a connection string using the local host, port `5432`, database `readinglist`, username `readinglist`, and the same password you gave Docker. This is the same general local-secrets approach documented in `jeemzu.api`. Never put a real password, API key, or other secret in a source file or commit it to GitHub.

A connection string is a semicolon-separated set of key/value pairs. Include the local host, port `5432`, database `readinglist`, username `readinglist`, and the password you chose for Docker.

5. Describe a book and the database

Create `Models/Book.cs`:

```
using System.ComponentModel.DataAnnotations;

namespace ReadingList.Api.Models;

public class Book
{
    public int Id { get; set; }

    [Required, StringLength(200)]
    public string Title { get; set; } = "";

    [Required, StringLength(200)]
    public string Author { get; set; } = "";

    public bool IsRead { get; set; }
}
```

Create `Data/AppDbContext.cs`:

```
using Microsoft.EntityFrameworkCore;
using ReadingList.Api.Models;

namespace ReadingList.Api.Data;

public class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    public DbSet<Book> Books => Set<Book>();
}
```

A **model** describes the information to store. The **DbContext** is EF Core's bridge between your C# code and the database. `jeemzu.api` uses this same basic EF Core pattern.

6. Add API endpoints

Create `Controllers/BooksController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ReadingList.Api.Data;
using ReadingList.Api.Models;

namespace ReadingList.Api.Controllers;

[ApiController]
[Route("api/books")]
public class BooksController(AppDbContext db) : ControllerBase
{
    [HttpGet]
    public async Task<List<Book>> GetBooks() =>
        await db.Books.OrderBy(book => book.Id).ToListAsync();

    [HttpPost]
    public async Task<ActionResult<Book>> AddBook(Book book)
    {
        book.Id = 0;
        db.Books.Add(book);
        await db.SaveChangesAsync();

        return StatusCode(StatusCodes.Status201Created, book);
    }

    [HttpPut("{id:int}")]
    public async Task<IActionResult> UpdateBook(int id, Book updatedBook)
    {
        var book = await db.Books.FindAsync(id);
        if (book is null) return NotFound();

        book.Title = updatedBook.Title;
        book.Author = updatedBook.Author;
        book.IsRead = updatedBook.IsRead;

        await db.SaveChangesAsync();
        return NoContent();
    }

    [HttpDelete("{id:int}")]
    public async Task<IActionResult> DeleteBook(int id)
    {
        var book = await db.Books.FindAsync(id);
        if (book is null) return NotFound();
```

```
        db.Books.Remove(book);
        await db.SaveChangesAsync();
        return NoContent();
    }
}
```

These routes cover **CRUD**: create, read, update, and delete.

7. Configure the API and database

Replace the contents of `Program.cs` with:

```
using Microsoft.EntityFrameworkCore;
using ReadingList.Api.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var frontendOrigin = builder.Configuration["Cors:AllowedOrigin"]
    ?? "http://localhost:5173";
builder.Services.AddCors(options =>
{
    options.AddPolicy("Frontend", policy =>
        policy.WithOrigins(frontendOrigin)
              .AllowAnyHeader()
              .AllowAnyMethod());
});

var app = builder.Build();

app.UseCors("Frontend");

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapControllers();
app.MapGet("/health", () => Results.Ok(new { status = "ok" }));

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await db.Database.MigrateAsync();
}

await app.RunAsync();
```

Create a migration and apply it to your local database:

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

A **migration** is a versioned description of a database change. This API also applies migrations when it starts, following the simple startup pattern documented in `jeemzu.api`. Later, when you change the model, create another migration so the database can catch up.

Start the API:

```
dotnet run
```

Use the HTTP address shown in the terminal (the generated project may use a different port) to open `/swagger`, for example `http://localhost:5050/swagger`. Swagger/OpenAPI lets you inspect and try the routes before the frontend is connected. Try `GET /api/books`, then send a `POST /api/books` body like:

```
{
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "isRead": false
}
```

Also check `/health`. Leave the API running while you work on the frontend.

8. Connect the frontend

Open a second terminal. From `reading-list/reading-list-web`, create `.env.local`:

```
VITE_API_URL=http://localhost:5050
```

Use the API's actual local address if it printed a different port. Vite reads `VITE_` variables when it starts, so restart its dev server after changing this file. Vite's `.env.local` is intended for local settings; do not commit it.

Replace `src/App.tsx` with:

```
import { useEffect, useState, type FormEvent } from "react";

type Book = {
  id: number;
  title: string;
  author: string;
  isRead: boolean;
};

const apiUrl = import.meta.env.VITE_API_URL ?? "http://localhost:5050";

export default function App() {
  const [books, setBooks] = useState<Book[]>([]);
  const [title, setTitle] = useState("");
  const [author, setAuthor] = useState("");
  const [error, setError] = useState("");

  async function loadBooks() {
    const response = await fetch(`${apiUrl}/api/books`);
    if (!response.ok) throw new Error("Could not load books.");
    setBooks(await response.json());
  }

  useEffect(() => {
    void loadBooks().catch(() => setError("Could not load books. Check that the API is running."));
  }, []);

  async function addBook(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");

    try {
      const response = await fetch(`${apiUrl}/api/books`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ title, author, isRead: false }),
      });
      if (!response.ok) throw new Error();
      setTitle("");
      setAuthor("");
      await loadBooks();
    } catch {
      setError("Could not add the book. Check that the API is running.");
    }
```

```
}

async function toggleRead(book: Book) {
  setError("");
  try {
    const response = await fetch(`${apiUrl}/api/books/${book.id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...book, isRead: !book.isRead }),
    });
    if (!response.ok) throw new Error();
    await loadBooks();
  } catch {
    setError("Could not update the book.");
  }
}

async function deleteBook(id: number) {
  setError("");
  try {
    const response = await fetch(`${apiUrl}/api/books/${id}`, {
      method: "DELETE",
    });
    if (!response.ok) throw new Error();
    await loadBooks();
  } catch {
    setError("Could not delete the book.");
  }
}

return (
  <main>
    <h1>My Reading List</h1>

    <form onSubmit={addBook}>
      <input
        required
        maxLength={200}
        placeholder="Book title"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />
      <input
        required
        maxLength={200}
```

```
          placeholder="Author"
          value={author}
          onChange={(event) => setAuthor(event.target.value)}
        />
        <button type="submit">Add book</button>
      </form>

      {error && <p role="alert">{error}</p>}

      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <span>
              {book.title} by {book.author} — {book.isRead ? "Read" : "Unread"}
            </span>{" "}
            <button type="button" onClick={() => void toggleRead(book)}>
              Mark {book.isRead ? "unread" : "read"}
            </button>{" "}
            <button type="button" onClick={() => void deleteBook(book.id)}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Start the frontend with `npm run dev` and open its Vite address. Add a book, mark it read, and delete it. Refresh the page: the list should still be there because PostgreSQL saved it.

The full trip is now working:

> React form → API request → ASP.NET controller → PostgreSQL → API response → React list

If your browser reports a CORS error, make sure `Cors:AllowedOrigin` in the API is the exact frontend origin, including `http://` or `https://` and the port. Restart the API after changing its setting.

9. Save your work safely on GitHub

You can create two GitHub repositories or keep both folders in one repository. Either way:

- Keep a `.gitignore` in each project. The Vite starter and .NET template include one; check that build output and local settings are ignored.

- Do not commit `.env.local`, database passwords, API keys, or other secrets.
- Commit small steps with messages that say what changed, then push to GitHub.

For example, from the `reading-list` folder:

```
git init
git add .
git status
git commit -m "Create reading list app"
```

Check the output of `git status` before committing. If you see a local settings file or credentials, stop and fix the ignore rules first. A secret that has already been pushed should be rotated; deleting it from a later commit is not enough.

10. Deploy the API and database to Render

The API repository uses a Docker-based ASP.NET Core service and a managed PostgreSQL database on Render. You can use the same shape for this small project:

1. Create a managed PostgreSQL database in Render. Keep the database and API service in the same region.
2. In the API project folder, create a `Dockerfile`:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_HTTP_PORTS=8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "ReadingList.Api.dll"]
```

3. Create a `.dockerignore` beside the Dockerfile:

```
bin/
obj/
.git/
```

4. Push the API project, Dockerfile, and `.dockerignore` to GitHub. In Render, create a **Web Service** from that repository and choose **Docker** as the runtime. Set the Dockerfile path if the file is not at the repository root.
5. Add environment variables to the Render service:

- `PORT` = `8080`
- `ASPNETCORE_HTTP_PORTS` = `8080`
- `ConnectionStrings__DefaultConnection` = a PostgreSQL connection string in Npgsql keyword format
- `Cors__AllowedOrigin` = your exact deployed Netlify site origin, such as `https://your-reading-list.netlify.app`

For the connection string, use Npgsql's keyword/value format: a semicolon-separated set of key/value pairs for the database's **internal** host, port, database name, username, and password. The API README notes that a Render `postgres://...` URL is not directly accepted by this setting; use the individual internal connection details instead. Keep the completed value in Render's environment settings, not in Git.

6. Deploy the service. Visit `https://YOUR-API.onrender.com/health`; you should see `{"status":"ok"}`. The API applies the first EF migration when it starts. Render may take a little while to start a new service.

The API Dockerfile and Render setup in `jeemzu.api` are useful references. This small project only deploys the .NET API and database; it does not need the reference repo's separate Python agents service.

11. Deploy the frontend to Netlify

1. Create a Netlify site from the frontend GitHub repository (or set the frontend subfolder as the base directory if using a monorepo).
2. Set the build command to `npm run build` and the publish directory to `dist`.
3. Add `VITE_API_URL` as a Netlify environment variable with the public Render API base URL, for example `https://YOUR-API.onrender.com`. Do not add `/api`; the frontend adds that path itself.
4. Deploy the site. Try adding, updating, and deleting a book from the live page.

If the site loads but cannot reach the API, check the Netlify `VITE_API_URL`, Render service logs, and the API's `Cors__AllowedOrigin`. CORS must be the exact public Netlify origin, without a trailing slash or API path. After changing an environment variable, redeploy the affected service.

The frontend README documents Netlify publishing the Vite `dist/` output as a static single-page app, and `jeemzu.me` sets its production API base with `VITE_API_URL`.

12. Where to go from here

When this version feels comfortable, pick one topic at a time:

- Explore React routes and reusable UI in `jeemzu.me` (it uses Wouter and Material UI).
- Look at the frontend's Zustand state patterns.
- Explore the API's EF Core models and migration history in `jeemzu.api`.
- Compare the API's Swagger/OpenAPI docs with the generated TypeScript API types in `jeemzu.me`.
- Look at the API repository's `Dockerfile` and Render Blueprint to learn how a larger deployment is described.

This tutorial intentionally has no accounts or user-specific lists. Before storing private information or letting different people manage separate lists, add authentication and authorization deliberately; the auth code in `jeemzu.api` is a more advanced reference. You can also stop here—one small app that works locally and online is a real accomplishment.

Quick summary: React + TypeScript displays the app, ASP.NET Core handles requests, EF Core talks to PostgreSQL, Netlify serves the frontend, and Render hosts the API and database. Build one layer at a time, keep secrets private, and test locally before deploying.